

REPLICATED COUNTER SERVICE

Communication, Logical Time, and Quorum Agreement

Course: Distributed Systems

Author: Nikita

Repository: <https://github.com/nikitapak162005-ai/nikita>

Date: 9 October 2026

Test Report

1. Overview

This report presents the implementation and testing of a replicated counter service developed in three stages: a gRPC counter service with RPC failure handling (Part A), Lamport logical-clock instrumentation (Part B), and a three-replica group using majority acknowledgements for writes (Part C). The report focuses on correctness, failure handling, logical ordering, and the performance cost of quorum writes.

2. Requirements Under Test

Task	Requirement	Verification evidence
A2	Increment applies its delta; idempotency suppresses duplicate mutations; retries reuse the same key.	test_increment_applies_delta; test_duplicate_key_not_reapplied; test_retry_after_timeout_is_safe
A2	Get on an unknown counter returns found = false.	test_get_missing_counter
A3	Concurrent clients produce the exact final total without lost updates.	test_concurrent_increments_exact
B1	Lamport time is carried on messages; receive uses $\max(\text{local}, \text{received}) + 1$.	logs/*.log; logs/trace_excerpt.txt
C1	2/3 acknowledgements commit; below-majority does not commit; replicas converge.	test_majority_commit_two_acks; test_no_commit_below_majority; test_replicas_converge
C3	Failure injection covers an unavailable replica, duplicate request, and timeout + retry.	test_majority_commit_two_acks; test_duplicate_request_moves_value_once; test_timeout_then_retry_commits_once

3. Test Environment

Item	Local environment
Operating System	Microsoft Windows 11 Pro
Python	3.14.7
grpcio	1.84.0
grpcio-tools	1.84.0
pytest	9.1.1
Execution	Localhost; automated tests use ephemeral ports; manual replica uses port 50051

4. Test Cases

All ten automated tests passed in the local environment. The table below summarizes the eight required tests; two additional failure-oriented tests are included in Section 5 and Appendix C.

Test	Purpose	Expected	Observed	Status	Evidence
test_increment_applies_delta	Apply one Increment.	new_value = 5	new_value = 5; duplicate = false	PASS	pytest
test_duplicate_key_not_reapplied	Repeat the same idempotency key.	value = 5; duplicate = true	value stayed 5; second reply marked duplicate	PASS	pytest
test_concurrent_increments_exact	2 × 1000 increments.	value = 2000	final value = 2000	PASS	pytest
test_get_missing_counter	Read unknown counter.	found = false; value = 0	found = false; value = 0	PASS	pytest
test_retry_after_timeout_is_safe	Timeout, then retry with same key.	counter changes once	retry succeeded; mutation applied once	PASS	pytest

Test	Purpose	Expected	Observed	Status	Evidence
test_majority_commit_two_acks	2 of 3 replicas acknowledge.	committed = true; acks = 2	committed = true; acks = 2	PASS	pytest
test_no_commit_below_majority	Only 1 of 3 acknowledges.	committed = false	not committed; reachable replica applied locally	PASS	pytest
test_replicas_converge	Run a batch of quorum writes.	all replicas equal	all replicas converged to the same value	PASS	pytest

5. Failure-Injection Results

Scenario	Injected fault	Expected invariant	Observed result	Evidence
Replica unavailable	One replica cannot be reached.	Write still commits with two acknowledgements; no caller exception.	Committed with 2/3 acknowledgements.	test_majority_commit_two_acks
Duplicate request	Same idempotency key is sent twice.	Counter changes once; second reply is duplicate.	Value remained 5; second reply returned was_duplicate = true.	test_duplicate_request_moves_value_once
Timeout + retry	First request is delayed past the deadline.	Retry succeeds with the same key; mutation occurs once.	First attempt timed out; retry succeeded; final value = 5.	test_timeout_then_retry_commits_once

Partial-write anomaly. With only one of three replicas reachable, the client reports the operation as not committed because one acknowledgement is below the majority of two. The reachable replica may still have applied the write locally. A consensus protocol such as Raft distinguishes replicated-but-uncommitted work from committed state through a replicated log and commit index.

6. Lamport Clock Analysis (Task B2)

Rules implemented. A local event increments the logical clock before the event: $L = L + 1$. When a process receives a timestamp T , it updates its clock as $L = \max(L, T) + 1$. Every outgoing message carries the sender's current Lamport value.

Happened-before. If $A \rightarrow B$, then $L(A) < L(B)$. The converse is not guaranteed: two events can have different Lamport values and still be concurrent when no local-order or message-delivery chain connects them.

Example from the real trace:

1. client-1 sends Increment(counter=x, delta=1) with Lamport time L=1.
2. replica-A receives the request and updates its clock to L=2.
3. replica-A applies the increment at L=3.
4. replica-A sends IncrementReply(new_value=1) with L=4.
5. client-1 receives the reply and updates its clock to L=5.

This sequence shows the happened-before relationship between the client request, server processing, server reply, and client receive event.

Causally ordered pair 1. client-1 SEND Increment(counter=x, delta=1) L=1 $\rightarrow$ replica-A RECV Increment(counter=x, delta=1) L=2. The SEND event happens-before the corresponding RECV event through the RPC message-delivery edge.

Causally ordered pair 2. replica-A SEND IncrementReply(new_value=1) L=4 → client-1 RECV IncrementReply(new_value=1) L=5. The server SEND event happens-before the client RECV event through the reply message.

Concurrent pair. client-1 SEND Increment(counter=x, delta=1) L=1 and client-2 SEND Increment(counter=y, delta=1) L=1 are independent client events. No local-order or message-delivery chain connects the two, so neither happened-before the other.

Limitation. Lamport clocks cannot identify concurrency from timestamp values alone. Vector clocks preserve additional causal information and can distinguish causally ordered events from concurrent events.

7. Performance Results (Task C4)

Each configuration executed 2,000 logical Increment operations. A quorum operation sends requests to three replicas but is counted as one logical operation. The p95 value represents the 95th-percentile latency.

Configuration	Median latency (ms)	p95 latency (ms)	Requests
Single replica, 1 client	1.090	2.672	2000
Single replica, 16 clients	15.543	22.506	2000
Quorum (3 replicas), 1 client	2.768	3.869	2000
Quorum (3 replicas), 16 clients	44.919	54.082	2000

Interpretation. Quorum writes were slower than single-replica writes in both workloads. With one client, median latency increased from 1.090 ms to 2.768 ms, while p95 increased from 2.672 ms to 3.869 ms. Under 16 concurrent clients, the gap was larger: median latency increased from 15.543 ms to 44.919 ms and p95 from 22.506 ms to 54.082 ms. The additional cost comes from coordinating several RPCs and waiting for a majority of acknowledgements; concurrency also increases contention for server threads and the shared state lock.

8. Known Limitations

1. **In-memory state:** Counter values are lost when a replica restarts.
2. **In-memory idempotency store:** Deduplication history is also lost on restart, so a replayed key could be applied again.
3. **No full consensus:** The service has no leader election, durable replicated log, commit index, or recovery/catch-up protocol.
4. **Replica divergence:** A replica that is unavailable during a write can miss updates.
5. **Stale reads:** Get reads from one replica, which may return an older value if that replica missed writes.

9. Reflection Questions

1. After a timeout, why can't the client tell "request lost" from "reply lost", and how does idempotency make this harmless?

A timeout only tells the client that no reply arrived before the deadline. The request may never have reached the server, or it may have been executed while the reply was lost or delayed. Reusing the same idempotency key on retry makes this ambiguity safe: if the first attempt already applied, the retry returns the stored result and does not apply the mutation again.

2. Your trace has a concurrent pair whose Lamport values differ. Why does the smaller value not mean that event happened first?

Lamport clocks guarantee that if $A \rightarrow B$, then $L(A) < L(B)$. They do not guarantee the reverse. For concurrent events with no causal chain between them, the clock values reflect each process's logical history rather than real-time order.

3. With three replicas and majority commit, which failures are tolerated, and what happens when two replicas fail?

One replica failure still leaves a majority of two, so new writes can be acknowledged by a quorum. With two failures, a majority is impossible and new writes cannot be reported as committed. The simplified implementation does not include the recovery and state-repair mechanisms of a full consensus protocol.

4. To survive replica restarts without losing committed writes, what is the smallest change, and which Week-6 mechanism does it anticipate?

The state and idempotency information must be stored durably before an acknowledgement is returned. A write-ahead log is one way to provide this persistence and is a step toward the durable replicated log used by consensus protocols such as Raft.

10. Conclusion

The assignment implemented and tested a replicated counter service using gRPC communication, idempotent request handling, retry logic, concurrency control, Lamport logical clocks, and majority-based writes across three replicas. All ten automated tests passed in the local environment. The performance measurements showed the additional latency introduced by quorum writes, especially under concurrent load. The project also makes the limitations of a simplified replicated design visible, including partial writes, stale replicas, and loss of in-memory state after restart.

Appendix A - Package Versions

```
colorama==0.4.6
grpcio==1.84.0
grpcio-tools==1.84.0
iniconfig==2.3.1
packaging==26.3
pluggy==1.6.0
protobuf==7.36.2
Pygments==2.21.0
pytest==9.1.1
setuptools==84.0.0
typing_extensions==4.16.0
```

Appendix B - Lamport Trace Excerpt

```
[client-1] SEND Increment(counter=x, delta=1) L=1
[client-2] SEND Increment(counter=y,delta=1) L=1
[client-3] SEND Get(counter=x) L=1
[replica-A] RECV Increment(counter=x, delta=1) L=2 (received L=1)
[replica-A] APPLY counter=x -> 1 L=3
[replica-A] SEND IncrementReply(new_value=1, duplicate=False) L=4
[client-1] RECV IncrementReply(new_value=1) L=5 (received L=4)
[client-1] SEND Increment(counter=x, delta=1) L=6
[replica-A] RECV Increment(counter=x, delta=1) L=7 (received L=6)
[replica-A] APPLY counter=x -> 2 L=8
[replica-A] SEND IncrementReply(new_value=2, duplicate=False) L=9
[client-1] RECV IncrementReply(new_value=2) L=10 (received L=9)
[replica-A] RECV Increment(counter=x, delta=1) L=10 (received L=1)
[replica-A] APPLY counter=x -> 3 L=11
[replica-A] SEND IncrementReply(new_value=3, duplicate=False) L=12
[client-2] RECV IncrementReply(new_value=1) L=13 (received L=12)
[client-1] RECV IncrementReply(new_value=3) L=13 (received L=12)
[client-1] SEND Increment(counter=x, delta=1) L=14
[client-2] SEND Increment(counter=y,delta=1) L=14
[replica-A] RECV Increment(counter=x, delta=1) L=15 (received L=14)
[replica-A] APPLY counter=x -> 4 L=16
[replica-A] SEND IncrementReply(new_value=4, duplicate=False) L=17
[client-1] RECV IncrementReply(new_value=4) L=18 (received L=17)
[client-2] RECV IncrementReply(new_value=2) L=18 (received L=17)
[replica-A] RECV Get(counter=x) L=18 (received L=1)
[replica-A] SEND GetReply(value=4, found=True) L=19
[client-3] RECV GetReply(value=4, found=True) L=20 (received L=19)
[client-3] SEND Get(counter=y) L=21
[replica-A] RECV Get(counter=y) L=22 (received L=21)
[replica-A] SEND GetReply(value=0, found=False) L=23
[client-3] RECV GetReply(value=0, found=False) L=24 (received L=23)
```

Appendix C - Automated Test Output

```
===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 --
C:\Users\user\Desktop\nikitaRSas2\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\user\Desktop\nikitaRSas2
collecting ... collected 10 items

tests/test_counter.py::test_increment_applies_delta PASSED [ 10%]
tests/test_counter.py::test_duplicate_key_not_reapplied PASSED [ 20%]
tests/test_counter.py::test_concurrent_increments_exact PASSED [ 30%]
tests/test_counter.py::test_get_missing_counter PASSED [ 40%]
tests/test_counter.py::test_retry_after_timeout_is_safe PASSED [ 50%]
tests/test_failures.py::test_majority_commit_two_acks PASSED [ 60%]
```

```
tests/test_failures.py::test_no_commit_below_majority PASSED [ 70%]
tests/test_failures.py::test_duplicate_request_moves_value_once PASSED [ 80%]
tests/test_failures.py::test_timeout_then_retry_commits_once PASSED [ 90%]
tests/test_failures.py::test_replicas_converge PASSED [100%]
```

```
===== 10 passed in 8.19s =====
```

Appendix D - Performance Benchmark Output

Configuration	Median(ms)	p95(ms)	Requests
Single replica, 1 client	1.090	2.672	2000
Single replica, 16 clients	15.543	22.506	2000
Quorum (3 replicas), 1 client	2.768	3.869	2000
Quorum (3 replicas), 16 clients	44.919	54.082	2000